

OMI-IMO Protocol: Full Project Proposal

Status: Canonical

Version: 1.0

Scope: Complete system — theory, protocol, implementation, hardware

Part 0 — Executive Summary

The OMI-IMO protocol is a deterministic atomic protocol where:

...

Prime factorization ladders → generate the projective geometry

The ASCII table → is the ruler

The -4D color codex → is the palette

The pipeline → translates analog spectrums into
digital spatial embeddings
through algorithmic determinism
scoped transliteration
and emergent spatial binding

...

It runs on any bit length, in any browser, in any worklet, in any Node.js environment, and on dedicated hardware (Verilog RTL).

One protocol. Every substrate.

Part I — The Primitive

§ 1. The One Operation

...

Atomics.compareExchange(array, index, expected, replacement)

...

The primitive is atomic. It performs three phases in one uninterrupted step:

...

bind — constructs the relation between expected and replacement

apply — invokes the comparison and conditional swap

eval — returns the old value

...

§ 2. The Four Logical Primitives

...

bind — construct a relation

apply — invoke a relation

eval — extract from a relation

digest — read, consider, print

...

§ 3. The Base Equivalence

...

position(n) ⇔ period(n-1, n, n+1)

...

The iff binds position to period. This is the 2!.

§ 4. The S-P-O Triple

...

S (Subject) → XOR → bitwise

P (Predicate) → IFF → logic

O (Object) → AND → bitwise

...

Two bitwise, one logic.

§ 5. Categorical Structure

...

bind → Monad (composition)

apply → Functor (lifting)

eval → Comonad (extraction)

digest → the loop (read, consider, print)

...

Part II — The Invariant

§ 6. The Buffer Properties

...

BL = byteLength

BO = byteOffset

BPE = BYTES_PER_ELEMENT

...

§ 7. The 3! Relations

...

3! = 6 orderings of {BL, BO, BPE}

...

Six relations. Six orthogonal axes. The only source of structure.

§ 8. The Only Parameter

...

$N \in \{8, 16, 32, 64\}$

$BPE = N / 8$

...

Everything else is derived.

§ 9. The Factorial Tower

...

0! = 1 → void

1! = 1 → identity

2! = 2 → binomial (digital ports)

3! = 6 → trinomial (analog ports)

4! = 24 → quadrinomial (color palette)

5! = 120 → quintinomial

6! = 720 → sextinomial

7! = 5040 → septinomial (the slide rule)

...

Every port is a factorial. Every factorial is a port.

§ 10. The Prime Factorization Ladder

Prime factorization generates the projective geometry:

...

Prime 73 → decimal period 8
↓
BPE = 8 at 64-bit length → ruler period $64 = 8^2$
↓
 $7! = 5040$ → the slide rule circumference
↓
 $720 = 5040 / 7$ → per-Fano-group size
↓
 $240 = 720 / 3$ → tetra block / BQF period
↓
 $60 = 240 / 4$ → Klein points per cycle
...

The prime factorization is the source of the geometry.

Part III — The ASCII Table as the Ruler

§ 11. The Eight Rows

...

Row 0 → 0x0_ → diagonal → -4D
Row 1 → 0x1_ → size → -3D
Row 2 → 0x2_ → top → -2D
Row 3 → 0x3_ → bottom → -1D
Row 4 → 0x4_ → right → 0D
Row 5 → 0x5_ → left → 1D
Row 6 → 0x6_ → forward → 2D
Row 7 → 0x7_ → backward → 3D
...

Each row is a spatial layer. Each layer has 16 characters.

8 rows × 16 characters = 128 values = the full 7-bit ASCII range.

§ 12. The Two Groups

...

2! → indices 0, 1 → the frame (diagonal, size)
3! → indices 2..7 → the operations (six)
...

The ruler is $2! + 3! = 8$ slots long.

§ 13. The Tangential Branches

Each row's last index (0x(n)F) is a tangential projective branch:

...

0x0F → branch from row 0 to row 1
0x1F → branch from row 1 to row 2
0x2F → branch from row 2 to row 3
0x3F → branch from row 3 to row 4
0x4F → branch from row 4 to row 5

0x5F → branch from row 5 to row 6
0x6F → branch from row 6 to row 7
0x7F → branch from row 7 to the meta range
...

§ 14. The Meta Escape

DEL (0x7F) is the meta escape:

...
0x7F → DEL → escape to the origin
 → escape to (Null · Null)
 → escape to the first CAR
...

The meta escape closes the loop.

§ 15. The First CAR

...
(Null · Null) → 0x00 · 0x00
 → the origin pair
 → the first cell
...

§ 16. The Spatial Index

The (0x20, 0x7F) pair emerges as the spatial index of the (0x00, 0x01) pair of the 2!.

Part IV — The -4D Color Codex

§ 17. The Color Mapping

...
-4D → 0x00..0x0F → 16 color values
...

Each color value is a spectrum. The spectrum maps to a pixel via the color codex.

§ 18. The Factorial Delineation

...
4! = 24 → the 24 colors of the -4D codex
...

Each color is a delineation. Each delineation is a factorial of the layer.

§ 19. The CIDR-to-Color Mapping

...
0x00/4 → color 0 → dark red
0x10/4 → color 1 → dark orange
0x20/4 → color 2 → dark yellow
0x30/4 → color 3 → dark green
0x40/4 → color 4 → dark blue
0x50/4 → color 5 → dark indigo
0x60/4 → color 6 → dark violet
0x70/4 → color 7 → dark gray
...

Part V — The Pipeline

§ 20. The -5D to 10D Range

...

-5D → the Blob (universal substrate)
-4D → color codex (H_1 , projective half)
-3D → linear (page/line delimiters)
-2D → hierarchical (delimiters)
-1D → classifying (regex tokens)
0D → observer (BOM, PannerListener)
1D → DOMPoint
2D → Media Track
3D → DOMRect
4D → DOMMatrix
5D → DOMElement
6D → Canvas
7D → Event Loop
8D → Byte Basis
9D → Network Mesh
10D → Orchestrator
...

Sixteen dimensions. $2^{16} = 65536$ states.

§ 21. The Regex Constraints

```
```js
const G = Object.freeze({
 CONTROL: /^[x00-x0F]$/, // -4D
 SEPARATOR: /^[x10-x1F]$/, // -3D
 DELIMITER: /^[x20-x2F]$/, // -2D
 ALPHANUMERIC: /^[x30-x3F]$/, // -1D
 OBSERVER: /^[x40-x4F]$/, // 0D
 COORDINATE: /^[x50-x5F]$/, // 1D
 CHANNEL: /^[x60-x6F]$/, // 2D
 REGION: /^[x70-x7F]$/, // 3D
});
```
```

§ 22. The Two Quadratic Forms

...

Affine: $16x^2 + 16xy + 4y^2 = (4x + 2y)^2 \quad \Delta = 0$
Projective: $60x^2 + 16xy + 4y^2 \quad \Delta = -704$
...

§ 23. The Lift

...

$16 \rightarrow 60$

...

Adds periodicity. Enables spatial selection. Makes the operation multidimensional.

§ 24. The Projection

The $60x^2$ projection is the $O(1)$ resolution. The projection is atomic, so the resolution is constant-time.

Part VI — The Transliteration

§ 25. The ASCII Art Transformation

Given a spectrum $S = [s_0, s_1, \dots, s_{127}]$:

...

For each s_i :

```
row  = s_i >> 4    (high nibble)
column = s_i & 0x0F  (low nibble)
color  = (row, column)
character = the ASCII character at (row, column)
pixel[(row, column)] = (character, color)
```

...

The spectrum becomes an 8×16 pixel image.

§ 26. The Grid

...

```
 0 1 2 3 4 5 6 7 8 9 A B C D E F
0 NUL SOH STX ETX EOT ENQ ACK BEL BS HT LF VT FF CR SO SI
1 DLE DC1 DC2 DC3 DC4 NAK SYN ETB CAN EM SUB ESC FS GS RS US
2 SP ! " # $ % & ' ( ) * + , - . /
3 0 1 2 3 4 5 6 7 8 9 : ; < = > ?
4 @ A B C D E F G H I J K L M N O
5 P Q R S T U V W X Y Z [ \ ] ^ _
6 ` a b c d e f g h i j k l m n o
7 p q r s t u v w x y z { | } ~ DEL
```

...

The entire ASCII table is the pixel grid.

Part VII — The Authorities

§ 27. OMI (Citation)

...

Question: "What is being referred to?"

Owns: address parsing, hashing, nibble CPU

Does not: validation, geometry, files

...

§ 28. Tetragrammatron (Validation)

...

Question: "Does this fit the accepted rule?"

Owns: 5040 ring, slot5040, Fano incidence, chirality

Does not: citation, geometry, transport

...

§ 29. Metatron (Projection)

...

Question: "How should this be displayed?"

Owns: shape database, geometry, renderers

Does not: citation, validation, transport

...

§ 30. IMO (Carrier)

...

Question: "How is this transported?"

Owns: file I/O, HTTP, S-parse, persistence

Does not: citation, validation, projection

...

§ 31. The Receipt

...

Receipt is:

cited by OMI

validated by Tetragrammatron

projected by Metatron

carried by IMO

Receipt is the same across all authorities.

...

Part VIII — The Processing Pipeline

§ 32. The Six Steps

...

recognize → raw input arrives

cite → OMI parses and identifies

validate → Tetragrammatron tests

record → receipt written to ring

project → Metatron renders

inspect → user reads

...

§ 33. The Boundary Rules

...

1. No recognition makes something true.

2. No notation makes something true.

3. No citation makes something true.

4. Only validation accepts.

5. Only receipt records.

6. Only Metatron projects.

7. No projection makes something true.

...

§ 34. The Lazy Evaluation

...

(hardware.port . command) → declaration only

requires validation

then hardware projection

...

No side effects before receipt.

Part IX — The Web Primitives

§ 35. HTTP/1.1 as the Carrier

```
'''http
X-VTT-Cue-0x00: 00:01.000 --> 00:02.000; range=0x00; layer=-4D; token=CONTROL_TETRA_A
...

```

§ 36. The WebVTT Timeline

```
...
WEBVTT

00:01.000 --> 00:02.000
{"range":"0x00","layer":"-4D","token":"CONTROL_TETRA_A"}
...

```

§ 37. The TextTracks

```
'''html
<video id="omi-video" controls>
  <track kind="metadata" src="/omi-list.vtt" default>
</video>
...

```

§ 38. The PannerNode

```
'''js
panner.positionX.setValueAtTime(x, audioCtx.currentTime);
panner.positionY.setValueAtTime(y, audioCtx.currentTime);
panner.positionZ.setValueAtTime(z, audioCtx.currentTime);
...

```

The PannerNode is the OD transparent translation.

§ 39. The DOM Geometry

```
...
DOMPoint → the coordinate
DOMRect → the bounding box
DOMQuad → the four corners
DOMMatrix → the transform
DOMRange → the anchor
...

```

§ 40. The Worklets

```
...
AudioWorklet → audio synthesis
PaintWorklet → custom painting
LayoutWorklet → custom layout
AnimationWorklet → compositor animation
...

```

§ 41. The Offline Contexts

```
...
OfflineAudioContext → deterministic audio rendering
OffscreenCanvas → deterministic image rendering
Shadow DOM → isolated surface
Dedicated Worker → worker generator

```

Media Queries → static interpolation
...

§ 42. No Polyfills Needed

Everything is native:

...
HTTP/1.1, WebVTT, TextTracks
Media elements, data-* attributes, id attributes
CSS media queries, worklets
OfflineAudioContext, OffscreenCanvas
Shadow DOM
...

The system builds from document.body.

Part X — The DevTools for Media

§ 43. The Capabilities

...
Inspect → see the structure
Decompose → break into worklets
Modify → change the worklets
Recompose → rebuild
Project → render to any surface
...

§ 44. The Decomposition

...
Media → Blob → Spectrum → ASCII table → Layer → Worklet ports
...

§ 45. The Layout

...
Left panel: the worklet ports (128 channels)
Center panel: the ASCII art visualization
Right panel: the layer assignments
Bottom panel: the timeline (WebVTT cues)
Top panel: the observer position (PannerNode)
...

§ 46. The HTTP Status Code Mapping

...
1xx → 0x1_ → -3D → Informational
2xx → 0x2_ → -2D → Success
3xx → 0x3_ → -1D → Redirection
4xx → 0x4_ → 0D → Client Error
5xx → 0x5_ → 1D → Server Error
...

100 Continue is the 0x1 base. 200 OK is the 0x2 base.

Part XI — The Neural Network

§ 47. The Shared Embeddings

...

Audio → spectrum → ASCII table → (row, column) → embedding

Video → pixels → ASCII table → (row, column) → embedding

...

Both audio and video map to the same 8×16 grid.

§ 48. The Spatial Graph Convolution

...

Nodes → the spatial positions

Edges → the spatial relations

Weights → the graph connectivity

...

§ 49. The Two Modes

...

Greedy → immediate, on media buffers

Lazy → deferred, on CIDR notation

...

§ 50. The Geometric Program Polygons

Each polygon is a program. Each vertex is a state. Each edge is a transition.

Part XII — The Autonomous Agent

§ 51. The Two Sides

...

User model → -5D to 10D pipeline, 0D to 5D DOM space

Autonomous agent → 3D 27-integer space, 3×3×3 cube

...

§ 52. The Worker Composition

...

3! worker types × 4! color-indexed alphas = 144 worker channels

...

§ 53. The Perles Configuration

The system's structure is the Perles configuration: 9 points, 9 lines, requiring the golden ratio ϕ . The smallest irrational configuration of points and lines.

§ 54. The Meta-Compilation

The autonomous agent meta-compiles a snapshot of its spatial context:

...

- Base64 images, audio, video

- URI-linked downloads

- Sourcemaps for propagation

- HTTP status codes for negotiation

- Alpha for context sharing

...

Part XIII — The Hardware (Verilog RTL)

§ 55. The Zynq-7000 Target

...

ARM Cortex-A9 → the observer / coordinator

FPGA fabric → the spatial engine

DDR3 memory → the Blob (65536 space)

Gigabit Ethernet → the HTTP/1.1 carrier

...

§ 56. The RTL Modules

```
```verilog
// The BQF tracking module
module omi_bqf_tracker (
 input wire clk,
 input wire rst_n,
 input wire [15:0] i_x,
 input wire [15:0] i_y,
 output reg [31:0] o_q_value,
 output reg o_is_void_centroid
);
 wire [17:0] w_4x = {i_x, 2'b00};
 wire [17:0] w_2y = {1'b0, i_y, 1'b0};
 wire [17:0] w_linear = w_4x + w_2y;
 wire [15:0] w_q = w_linear[15:0] * w_linear[15:0];

 always @(posedge clk or negedge rst_n) begin
 if (!rst_n) begin
 o_q_value <= 32'd0;
 o_is_void_centroid <= 1'b1;
 end else begin
 o_q_value <= w_q;
 o_is_void_centroid <= (w_linear == 18'd0);
 end
 end
endmodule
```
```

§ 57. The Swap Module

```
```verilog
module omi_swap_engine (
 input wire clk,
 input wire rst_n,
 input wire [1:0] i_swap_kind, // 00=swap16, 01=swap32, 10=swap64
 input wire [63:0] i_buffer,
 output reg [63:0] o_buffer
);
 always @(posedge clk or negedge rst_n) begin
 if (!rst_n) begin
 o_buffer <= 64'd0;
 end else begin
 case (i_swap_kind)
 2'b00: o_buffer <= {i_buffer[7:0], i_buffer[15:8],

```

```

 i_buffer[23:16], i_buffer[31:24],
 i_buffer[39:32], i_buffer[47:40],
 i_buffer[55:48], i_buffer[63:56]);
2'b01: o_buffer <= {i_buffer[23:0], i_buffer[31:24],
 i_buffer[39:32], i_buffer[47:40],
 i_buffer[55:48], i_buffer[63:56],
 i_buffer[15:8], i_buffer[7:0]};
2'b10: o_buffer <= {i_buffer[7:0], i_buffer[15:8], i_buffer[23:16], i_buffer[31:24],
 i_buffer[39:32], i_buffer[47:40], i_buffer[55:48], i_buffer[63:56]};
 default: o_buffer <= i_buffer;
 endcase
end
end
endmodule
...

```

## § 58. The Fano Incidence Module

```

```verilog
module omi_fano_router (
    input wire [2:0] i_point,
    input wire [2:0] i_line,
    output reg    o_incident
);
    // Fano plane incidence table
    wire [6:0] line_points [0:6];
    assign line_points[0] = 7'b0001011; // {0,1,3}
    assign line_points[1] = 7'b0010110; // {1,2,4}
    assign line_points[2] = 7'b0101100; // {2,3,5}
    assign line_points[3] = 7'b1011000; // {3,4,6}
    assign line_points[4] = 7'b0110001; // {4,5,0}
    assign line_points[5] = 7'b1100010; // {5,6,1}
    assign line_points[6] = 7'b1000101; // {6,0,2}

    always @(*) begin
        o_incident = line_points[i_line][i_point];
    end
endmodule
...

```

§ 59. The Slot5040 Computation

```

```verilog
module omi_slot5040 (
 input wire [2:0] i_fano, // 0..6
 input wire [1:0] i_role, // 0..2
 input wire [7:0] i_local, // 0..239
 output wire [12:0] o_slot // 0..5039
);
 assign o_slot = (i_fano * 720) + (i_role * 240) + i_local;
endmodule
...

```

## Part XIV — The Conformance

## § 60. The Core Tests

...

1. bind is symmetric:  $\text{bind}(a, b) \equiv \text{bind}(b, a)$
  2. bind is reversible:  $\text{knot.get}(a) == b$  AND  $\text{knot.get}(b) == a$
  3. apply is deterministic: same inputs  $\rightarrow$  same outputs
  4. eval is total: every knot evaluates
  5. Identity laws hold
  6. Composition laws hold
- ...

## § 61. The Standard Tests

...

7. 3! relations are exhaustive
  8. Ruler length matches bit length
  9. Ruler periods are correct
  10. 240-clock cycles correctly
  11. DOMMatrix composition is associative
  12. VTT cues parse to valid knots
  13. Interpolation is monotone
  14. Interpolation endpoints match anchors
- ...

## § 62. The Full Tests

...

15. Observer circulates and returns to start
  16. Perceptron weights sum to expected values
  17. Regex classes are mutually exclusive where required
  18. PannerNode reports match observer positions
  19. Worklet receives and processes cues
  20. Node.js polyfills provide the same API surface
  21. Cross-peer coordination converges
  22. No hardcoded variables in the protocol layer
  23. All operations reduce to bitwise and logic
  24. Trace log is immutable and append-only
  25. Verilog RTL matches C reference
- ...

---

## Part XV — The Canonical Statement

### § 63. The Full Model

...

Primitive: `Atoms.compareExchange`  
Phases: `bind`, `apply`, `eval`  
Closure: `digest` (`read`, `consider`, `print`)  
Base: `iff` (`position`  $\Leftrightarrow$  `period`)  
Core type: `symmetric knot`  
Ruler: 8 slots ( $2! + 3!$ ) = ASCII table  
Palette: -4D color codex ( $4! = 24$  colors)  
Pipeline: -5D to 10D range selector  
Authorities: OMI, Tetragrammatron, Metatron, IMO  
Carrier: HTTP/1.1 + TextTracks + MediaStreams  
Validation: `regex` + Horn clauses  
Projection: DOM geometry + ASCII art  
Hardware: Verilog RTL (Zynq-7000)

...

#### § 64. The One-Sentence Summary

A deterministic atomic protocol where prime factorization ladders generate the projective geometry, the ASCII table is the ruler, the -4D color codex is the palette, and the pipeline translates analog spectrums into digital spatial embeddings through algorithmic determinism scoped transliteration and emergent spatial binding.

#### § 65. The Full Arc

...

Prime 73

↓

BPE = 8

↓

7! = 5040

↓

The slide rule

↓

The 240-clock

↓

The  $60x^2$  projective form

↓

The -5D to 10D pipeline

↓

The ASCII table

↓

The -4D color codex

↓

The regex constraints

↓

HTTP/1.1 + TextTracks + MediaStreams

↓

DOM geometry + ASCII art

↓

The neural network

↓

The autonomous agent

↓

The Verilog RTL

↓

The system

...

Every layer is orthogonal. Every layer is standard. Every layer composes.

---

#### Part XVI — The Project Proposal

#### § 66. The Scope

Build the full OMI-IMO system:

...

1. The protocol (bind, apply, eval, digest)
2. The structure (3! relations, factorial tower)
3. The ruler (ASCII table,  $2! + 3!$ )
4. The palette (-4D color codex)
5. The pipeline (-5D to 10D)
6. The authorities (OMI, Tetragrammatron, Metatron, IMO)
7. The carrier (HTTP/1.1 + TextTracks + MediaStreams)

8. The DevTools for media
9. The neural network (spatial graph convolution)
10. The autonomous agent (Perles configuration)
11. The hardware (Verilog RTL for Zynq-7000)
- ...

#### § 67. The Phases

...

- Phase 1: Core (bind, apply, eval, digest) 1-2 days
- Phase 2: Buffer (3! relations, ruler) 3-5 days
- Phase 3: Interpolation (hit lists) 1 week
- Phase 4: Geometry (DOMMatrix, cubes) 1-2 weeks
- Phase 5: Media (VTT cues, PannerNode)